

GridSearchCV#

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Description:

Exhaustive search over specified parameter values for an estimator. Important members are fit, predict. GridSearchCV implements a “fit” and a “score” method. It also implements “score_samples”, “predict”, “predict_proba”, “decision_function”, “transform” and “inverse_transform” if they are implemented in the estimator used. The parameters of the estimator used to apply these methods are optimized by cross-validated grid-search over a parameter grid.

Function Signature

```
class sklearn.model_selection.GridSearchCV(estimator,  
param_grid, *, scoring=None, n_jobs=None, refit=True,  
cv=None, verbose=0, pre_dispatch='2*n_jobs',  
error_score=nan, return_train_score=False)[source]#
```

Parameters:

`estimator`estimator object

`param_grid`dict or list of dictionaries

`scoring`str, callable, list, tuple or dict, default=None

```
class sklearn.model_selection.GridSearchCV(estimator,  
param_grid, *, scoring=None, n_jobs=None, refit=True,  
cv=None, verbose=0, pre_dispatch='2*n_jobs',  
error_score=nan, return_train_score=False)[source]#
```

Parameters:

`estimator`estimator object

`param_grid`dict or list of dictionaries

`scoring`str, callable, list, tuple or dict, default=None

`decision_function(X)`[source]#

Parameters:

`X`indexable, length `n_samples`

Returns:

`y_score`ndarray of shape `(n_samples,)` or `(n_samples, n_classes)` or `(n_samples, n_classes * (n_classes-1) / 2)`

Parameters

`param_grid` or list of dictionaries

Dictionary with parameters names (str) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.

`cv`, `cross`

Determines the cross-validation splitting strategy. Possible inputs for `cv` are: `None`, to use the default 5-fold cross validation, integer, to specify the number of folds in a (Stratified)KFold, CV splitter, An iterable yielding (train, test) splits as arrays of indices. For integer/`None` inputs, if the estimator is a classifier and `y` is either binary or multiclass, StratifiedKFold is used. In all other cases, KFold is used. These splitters are instantiated with `shuffle=False` so the splits will be the same across calls. Refer User Guide for the various cross-validation strategies that can be used here. Changed in version 0.22: `cv` default value if `None` changed from 3-fold to 5-fold.

`best_params_`

Parameter setting that gave the best results on the hold out data. For multi-metric evaluation, this is present only if `refit` is specified.

`y_scorendarray` of shape `(n_samples,)` or `(n_samples, n_classes)` or `(n_samples, n_classes * (n_classes`

Result of the decision function for X based on the estimator with the best found parameters.

`Xarray`

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of features. For precomputed kernel or distance matrix, the expected shape of X is `(n_samples, n_samples)`.

`yarray`

Target relative to X for classification or regression; None for unsupervised learning.

`**paramsdict of str`

Parameters passed to the fit method of the estimator, the scorer, and the CV splitter. If a fit parameter is an array-like whose length is equal to `num_samples` then it will be split by cross-validation along with X and y. For example, the `sample_weight` parameter is split because `len(sample_weights) = len(X)`. However, this behavior does not apply to groups which is passed to the splitter configured via the `cv` parameter of the constructor. Thus, `groups` is used to perform the split and determines which samples are assigned to the each side of the a split.

`paramsdict`

Parameter names mapped to their values.

`Xarray`

Input data, where `n_samples` is the number of samples and `n_features` is the number of features.

`yarray`

Target relative to `X` for classification or regression; `None` for unsupervised learning.

`**paramsdict`

Parameters to be passed to the underlying scorer(s). Added in version 1.4: Only available if `enable_metadata_routing=True`. See [Metadata Routing User Guide](#) for more details.

`**paramsdict`

Estimator parameters.

Code Examples

```

{
'param_kernel': masked_array(data = ['poly', 'poly', 'rbf',
'rbf'],
                                mask = [False False False False]...)
'param_gamma': masked_array(data = [-- -- 0.1 0.2],
                                mask = [ True  True False False]...),
'param_degree': masked_array(data = [2.0 3.0 -- --],
                                mask = [False False  True
True]...),
'split0_test_score' : [0.80, 0.70, 0.80, 0.93],
'split1_test_score' : [0.82, 0.50, 0.70, 0.78],
'mean_test_score' : [0.81, 0.60, 0.75, 0.85],
'std_test_score' : [0.01, 0.10, 0.05, 0.08],
'rank_test_score' : [2, 4, 3, 1],
'split0_train_score' : [0.80, 0.92, 0.70, 0.93],
'split1_train_score' : [0.82, 0.55, 0.70, 0.87],
'mean_train_score' : [0.81, 0.74, 0.70, 0.90],
'std_train_score' : [0.01, 0.19, 0.00, 0.03],
'mean_fit_time' : [0.73, 0.63, 0.43, 0.49],
'std_fit_time' : [0.01, 0.02, 0.01, 0.01],
'mean_score_time' : [0.01, 0.06, 0.04, 0.04],
'std_score_time' : [0.00, 0.00, 0.00, 0.01],
'params' : [{'kernel': 'poly', 'degree': 2}, ...],
}

```

```

>>> from sklearn import svm, datasets
>>> from sklearn.model_selection import GridSearchCV
>>> iris = datasets.load_iris()
>>> parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
>>> svc = svm.SVC()
>>> clf = GridSearchCV(svc, parameters)
>>> clf.fit(iris.data, iris.target)
GridSearchCV(estimator=SVC(),
              param_grid={'C': [1, 10], 'kernel': ('linear',
'rbf')})
>>> sorted(clf.cv_results_.keys())
['mean_fit_time', 'mean_score_time', 'mean_test_score',...
 'param_C', 'param_kernel', 'params',...
 'rank_test_score', 'split0_test_score',...
 'split2_test_score', ...
 'std_fit_time', 'std_score_time', 'std_test_score']

```

Notes & Warnings

NOTE See also `ParameterGrid` Generates all the combinations of a hyperparameter grid. `train_test_split` Utility function to split the data into a development set usable for fitting a `GridSearchCV` instance and an evaluation set for its final evaluation. `sklearn.metrics.make_scorer` Make a scorer from a performance metric or loss function.

Detailed Documentation

Documentation

Exhaustive search over specified parameter values for an estimator.

Important members are `fit`, `predict`.

`GridSearchCV` implements a “fit” and a “score” method. It also implements “score_samples”, “predict”, “predict_proba”, “decision_function”, “transform” and “inverse_transform” if they are implemented in the estimator used.

The parameters of the estimator used to apply these methods are optimized by cross-validated grid-search over a parameter grid.

Read more in the User Guide.

This is assumed to implement the scikit-learn estimator interface. Either estimator needs to provide a score function, or scoring must be passed.

Dictionary with parameters names (str) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.

Strategy to evaluate the performance of the cross-validated model on the test set.

If scoring represents a single score, one can use:

a single string (see String name scorers);

a callable (see Callable scorers) that returns a single value;

None, the estimator’s default evaluation criterion is used.

If scoring represents multiple scores, one can use:

a list or tuple of unique strings;

a callable returning a dictionary where the keys are the metric names and the values are the metric scores;

a dictionary with metric names as keys and callables as values.

See [Specifying multiple metrics for evaluation](#) for an example.

Number of jobs to run in parallel. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See [Glossary](#) for more details.

Changed in version v0.20: `n_jobs` default changed from 1 to None

Refit an estimator using the best found parameters on the whole dataset.

For multiple metric evaluation, this needs to be a str denoting the scorer that would be used to find the best parameters for refitting the estimator at the end.

Where there are considerations other than maximum score in choosing a best estimator, `refit` can be set to a function which returns the selected `best_index_` given `cv_results_`. In that case, the `best_estimator_` and `best_params_` will be set according to the returned `best_index_` while the `best_score_` attribute will not be available.

The refitted estimator is made available at the `best_estimator_` attribute and permits using `predict` directly on this `GridSearchCV` instance.

Also for multiple metric evaluation, the attributes `best_index_`, `best_score_` and `best_params_` will only be available if `refit` is set and all of them will be determined w.r.t this specific scorer.

See scoring parameter to know more about multiple metric evaluation.

See Custom refit strategy of a grid search with cross-validation to see how to design a custom selection strategy using a callable via refit.

See this example for an example of how to use refit=callable to balance model complexity and cross-validated score.

Changed in version 0.20: Support for callable added.

Determines the cross-validation splitting strategy. Possible inputs for cv are:

None, to use the default 5-fold cross validation,

integer, to specify the number of folds in a (Stratified)KFold,

An iterable yielding (train, test) splits as arrays of indices.

For integer/None inputs, if the estimator is a classifier and y is either binary or multiclass, StratifiedKFold is used. In all other cases, KFold is used. These splitters are instantiated with shuffle=False so the splits will be the same across calls.

Refer User Guide for the various cross-validation strategies that can be used here.

Changed in version 0.22: cv default value if None changed from 3-fold to 5-fold.

Controls the verbosity: the higher, the more messages.

>1 : the computation time for each fold and parameter candidate is displayed;

>2 : the score is also displayed;

>3 : the fold and candidate parameter indexes are also displayed together with the

starting time of the computation.

Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs

An int, giving the exact number of total jobs that are spawned

A str, giving an expression as a function of `n_jobs`, as in `'2*n_jobs'`

Value to assign to the score if an error occurs in estimator fitting. If set to `'raise'`, the error is raised. If a numeric value is given, `FitFailedWarning` is raised. This parameter does not affect the refit step, which will always raise the error.

If `False`, the `cv_results_` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance.

Added in version 0.19.

Changed in version 0.21: Default value was changed from `True` to `False`

A dict with keys as column headers and values as columns, that can be imported into a pandas `DataFrame`.

For instance the below given table

will be represented by a `cv_results_` dict of:

The key 'params' is used to store a list of parameter settings dicts for all the parameter candidates.

The `mean_fit_time`, `std_fit_time`, `mean_score_time` and `std_score_time` are all in seconds.

For multi-metric evaluation, the scores for all the scorers are available in the `cv_results_` dict at the keys ending with that scorer's name ('_<scorer_name>') instead of '_score' shown above. ('split0_test_precision', 'mean_train_precision' etc.)

Estimator that was chosen by the search, i.e. estimator which gave highest score (or smallest loss if specified) on the left out data. Not available if `refit=False`.

See `refit` parameter for more information on allowed values.

Mean cross-validated score of the best_estimator

For multi-metric evaluation, this is present only if `refit` is specified.

This attribute is not available if `refit` is a function.

Parameter setting that gave the best results on the hold out data.

For multi-metric evaluation, this is present only if `refit` is specified.

The index (of the `cv_results_` arrays) which corresponds to the best candidate parameter setting.

The dict at `search.cv_results_['params'][search.best_index_]` gives the parameter setting for the best model, that gives the highest mean score (`search.best_score_`).

For multi-metric evaluation, this is present only if refit is specified.

Scorer function used on the held out data to choose the best parameters for the model.

For multi-metric evaluation, this attribute holds the validated scoring dict which maps the scorer key to the scorer callable.

The number of cross-validation splits (folds/iterations).

Seconds used for refitting the best model on the whole dataset.

This is present only if refit is not False.

Added in version 0.20.

Whether or not the scorers compute several metrics.

Number of features seen during fit.

Names of features seen during fit. Only defined if `best_estimator_` is defined (see the documentation for the `refit` parameter for more details) and that `best_estimator_` exposes `feature_names_in_` when fit.

Added in version 1.0.

Generates all the combinations of a hyperparameter grid.

Utility function to split the data into a development set usable for fitting a `GridSearchCV` instance and an evaluation set for its final evaluation.

Make a scorer from a performance metric or loss function.

The parameters selected are those that maximize the score of the left out data, unless

an explicit score is passed in which case it is used instead.

If `n_jobs` was set to a value higher than one, the data is copied for each point in the grid (and not `n_jobs` times). This is done for efficiency reasons if individual jobs take very little time, but may raise errors if the dataset is large and not enough memory is available. A workaround in this case is to set `pre_dispatch`. Then, the memory is copied only `pre_dispatch` many times. A reasonable value for `pre_dispatch` is $2 * n_jobs$.

Call `decision_function` on the estimator with the best found parameters.

Only available if `refit=True` and the underlying estimator supports `decision_function`.

Must fulfill the input assumptions of the underlying estimator.

Result of the decision function for `X` based on the estimator with the best found parameters.

Run fit with all sets of parameters.

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of features. For precomputed kernel or distance matrix, the expected shape of `X` is `(n_samples, n_samples)`.

Target relative to `X` for classification or regression; None for unsupervised learning.

Parameters passed to the fit method of the estimator, the scorer, and the CV splitter.

If a fit parameter is an array-like whose length is equal to `num_samples` then it will be split by cross-validation along with `X` and `y`. For example, the `sample_weight` parameter is split because `len(sample_weights) = len(X)`. However, this behavior does not apply to groups which is passed to the splitter configured via the `cv` parameter of the constructor. Thus, `groups` is used to perform the split and determines which samples are assigned to

the each side of the a split.

Instance of fitted estimator.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

Added in version 1.4.

A MetadataRouter encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Call `inverse_transform` on the estimator with the best found params.

Only available if the underlying estimator implements `inverse_transform` and `refit=True`.

Must fulfill the input assumptions of the underlying estimator.

Result of the `inverse_transform` function for X based on the estimator with the best found parameters.

Call `predict` on the estimator with the best found parameters.

Only available if `refit=True` and the underlying estimator supports `predict`.

Must fulfill the input assumptions of the underlying estimator.

The predicted labels or values for X based on the estimator with the best found parameters.

Call `predict_log_proba` on the estimator with the best found parameters.

Only available if `refit=True` and the underlying estimator supports `predict_log_proba`.

Must fulfill the input assumptions of the underlying estimator.

Predicted class log-probabilities for X based on the estimator with the best found parameters. The order of the classes corresponds to that in the fitted attribute `classes_`.

Call `predict_proba` on the estimator with the best found parameters.

Only available if `refit=True` and the underlying estimator supports `predict_proba`.

Must fulfill the input assumptions of the underlying estimator.

Predicted class probabilities for X based on the estimator with the best found parameters. The order of the classes corresponds to that in the fitted attribute `classes_`.

Return the score on the given data, if the estimator has been refit.

This uses the score defined by `scoring` where provided, and the `best_estimator_.score` method otherwise.

Input data, where `n_samples` is the number of samples and `n_features` is the number of features.

Target relative to X for classification or regression; None for unsupervised learning.

Parameters to be passed to the underlying scorer(s).

Added in version 1.4: Only available if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details.

The score defined by scoring if provided, and the `best_estim`